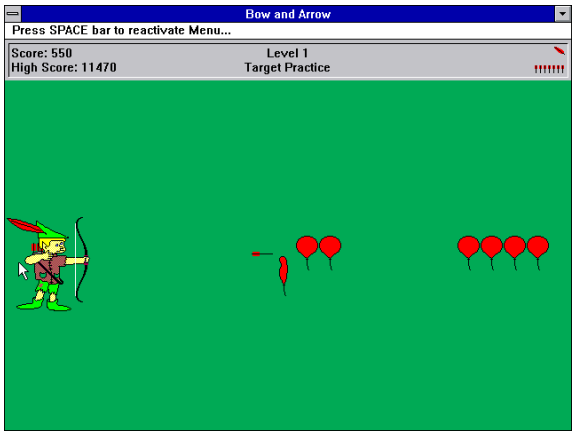
Práctica 1: Arco & Flecha 1.0

Curso 2026-2027. Tecnología de la Programación de Videojuegos 1. UCM

Fecha de entrega: 26 de octubre de 2026



En esta práctica implementaremos una versión particular del clásico juego *Bow & Arrow* de 1992 para Windows 3.1. Es un juego sencillo en el que el jugador debe explotar los globos que suben por la pantalla disparando flechas con un arco. Se utilizará C++ y la biblioteca SDL para manejar la entrada/salida del juego. La funcionalidad básica de nuestra versión del juego consiste en:

- A lo ancho de la pantalla van apareciendo por el borde inferior, uno a uno, globos de colores variados que ascienden a distintas velocidades hasta desaparecer por el borde superior.
- Junto al borde izquierdo de la pantalla se visualiza un arco apuntando hacia la derecha. El jugador controla este arco, que se puede mover en vertical y con el que se pueden lanzar flechas. La posición vertical del arco se controla con las teclas ↑ y ↓, con ← se carga una flecha y con → se lanza la flecha cargada.
- Una vez lanzada, la flecha se mueve en horizontal de izquierda a derecha hasta desaparecer por el extremo derecho de la ventana. Si la punta de la fecha impacta con un globo, este explota y otorga una puntuación al jugador. La flecha continúa su camino, en el que puede explotar más globos.
- La puntuación y el número de flechas disponibles deben mostrarse de alguna manera y actualizarse correspondientemente (al menos en la consola y opcionalmente en la ventana gráfica).
- El juego se acaba cuando se acaban las flechas, inicialmente 10.

En prácticas sucesivas o en los apartados opcionales se extenderá el juego con otras funcionalidades.

Detalles de implementación

Esta sección describe la funcionalidad esperada de la práctica y las directrices que has de seguir para implementarla. Lee su contenido con atención.

Clases proporcionadas

En el campus virtual, además de la plantilla para proyectos con SDL3, se proporcionan unas clases que serán útiles para implementar el juego:

- `Texture` (archivos `texture.h/cpp`) encapsula el manejo de las texturas de la SDL, posiblemente organizadas como rejillas de frames. Guarda un puntero a la textura SDL e información sobre su tamaño total y el tamaño de sus frames. Esta clase proporciona métodos para construir/cargar la textura de un fichero, para dibujarla en la posición proporcionada, bien en su totalidad (método `render`) o bien uno de sus frames (método `renderFrame`), y para destruirla.
- `Rectangle` (archivos `rectangle.h/.cpp`) representa un rectángulo alineado con los ejes en el plano 2D y proporciona métodos para comprobar intersecciones y otras operaciones habituales. Se proporciona para mantener desacoplada la lógica del juego de la vista, aunque la clase `SDL_FRect` de SDL y sus funciones asociadas son equivalentes.

Consulta las declaraciones y comentarios en las cabeceras `texture.h` y `rectangle.h` para conocer los métodos que proporcionan y su utilidad.

Diseño de clases

A continuación se indican las clases y métodos que debes implementar obligatoriamente. A algunos métodos se les da un nombre específico para poder referirnos a ellos en otras partes del texto. Además, deberás implementar los métodos (y posiblemente las clases) adicionales que consideres necesarios para mejorar la claridad y reusabilidad del código. Cada clase (salvo la plantilla `Vector2D` y su alias `Point2D`) se corresponderá con un par de archivos `.h` y `.cpp`.

Clase	Descripción	Atributos (📦) y métodos (⚙)
 `Vector2D`	Vector o punto en el plano bidimensional. Es una plantilla `template<std::floating_point T = float> // por defecto` sobre el tipo `T` de sus coordenadas. Además, en el mismo archivo `vector2D.h`, declara con `using` un alias `Point2D`. Este nombre se usará cuando nos refiramos a un punto en lugar de una dirección.	📦 Dos coordenadas del tipo genérico `T` ⚙ `T getX() const` y `T getY() const` ⚙ Operadores `+`, `-`, `*` (producto escalar de dos vectores), `*` (producto por un escalar), y otros que hagan falta. ⚙ `T length() const` para obtener la longitud del vector.
 `Balloon`	Globo que recorre la ventana de abajo a arriba a una velocidad fija. Hay globos de siete colores distintos. En § *Generación aleatoria de globos* se explica cómo se generan aleatoriamente estos globos. Al ser alcanzado por la flecha, el globo explota con una animación que itera con un cierto retardo por las columnas de la textura.	📦 Su posición y dimensiones, de tipo `Rectangle` o con atributos separados 📦 Su color (índice de fila en la textura) 📦 Su velocidad, de tipo `Vector2D` ⚙ `bool hit(const Rectangle& obj)` para comprobar colisiones. Más información en § *Manejo de colisiones*.
 `Arrow`	Flecha que lanza el jugador y que destruye a todos los globos que toca con su punta mientras se mueve hacia la derecha de la pantalla.	📦 Un puntero al `Game` (para las colisiones) 📦 Su posición y dimensiones 📦 Su velocidad
 `Bow`	Arco que lanza flechas desde el extremo izquierdo de la pantalla y que se puede mover en vertical con las teclas de dirección. Es necesario armar el arco (colocar la flecha) antes de cada disparo.	📦 Su posición y dimensiones 📦 Su velocidad o dirección de movimiento 📦 Número de flechas disponibles 📦 Si el arco está armado o no ⚙ `void handleEvent(const SDL_Event& event)` para manejar los eventos
`Game`	Controla la lógica y la renderización del juego como se ha explicado en clase. Su constructor debe inicializar SDL, crear una ventana, cargar las texturas estáticas, leer el archivo de configuración y crear los objetos del juego, etc. Su destructor liberará todos los recursos reservados.	📦 Punteros a la ventana y al renderizador 📦 Un puntero o vector de punteros (tipo `vector`) para cada tipo de objeto del juego 📦 Booleano `exit` del bucle de juego 📦 Array de texturas (véase más adelante) ⚙ Métodos `render`, `update` y `handleEvents` del bucle de juego ⚙ `checkCollision` (ver más adelante)

Además, todos los objetos que representan elementos del juego guardarán un puntero a su textura de tipo `Texture` y dispondrán de los siguientes métodos:

- `void render() const` que renderiza el objeto en la ventana de acuerdo a su forma y textura. Más detalles en § *Renderizado del juego*.
- `bool update(float delta)` para moverse o actualizarse en cada ciclo del juego. Más detalles en § *Movimiento de los objetos*. El método devuelve un booleano que indica si el objeto debe mantenerse (`true`) o ser eliminado (`false`), como se explica en § *Eliminación de los objetos del juego*.

Carga de texturas

La mayoría de las texturas que utilizaremos son conocidas de antemano y deben cargarse durante la inicialización del juego y guardarse en un array estático de tipo `array<Texture*, NUM_TEXTURES>` que aparece en el archivo `Game.cpp` de la plantilla. El método público `Texture* getTexture(TextureName) const` de `Game` permitirá acceder a las texturas cuando hayan de ser usadas. Sigue las instrucciones de la plantilla para especificar las texturas y cargarlas.

Renderizado del juego

El bucle principal del juego (método `Game::run`) invoca al método `Game::render`, que

1. borra la pantalla con `SDL_RenderClear`,
2. delega el renderizado a los diferentes objetos del juego llamando a sus respectivos métodos `render` y
3. presenta la escena en la pantalla llamando a la función `SDL_RenderPresent`.

Cada objeto del juego sabe cómo pintarse, pues conoce su propia posición, forma y textura. Por lo tanto, cada cual construirá su rectángulo de destino e invocará al método `render` o `renderFrame` de la textura correspondiente, que realizará el renderizado real.

Movimiento de los objetos

El bucle principal del juego (método `Game::run`) desencadena periódicamente actualizaciones de su estado llamando al método `Game::update`. Este a su vez llama sucesivamente a los métodos `update` de cada uno de los elementos del juego. Son ellos los que conocen dónde se encuentran y cómo deben moverse. En esta práctica hay dos tipos de movimiento esencialmente distintos.

Por un lado, cada globo o flecha tiene una velocidad asignada y se mueve en cada ciclo aplicándola (multiplicada por el tiempo trascurrido) a su posición. Cuando cualquiera de estos objetos desaparece por un extremo de la pantalla, se destruye y libera su memoria reservada. La flecha se crea desde el arco con una velocidad inicial fija y varias flechas pueden coexistir en la escena. Los globos se crean como se explica en § *Generación aleatoria de globos*.

Por otro lado, el usuario controla el movimiento del arco a través del teclado. Mientras estén pulsadas las flechas verticales de dirección (u otras teclas equivalentes), el arco se desplaza en dicha dirección de la escena. El movimiento se implementará de la siguiente forma: cuando se pulsa o suelta una tecla de desplazamiento, el bucle de manejo de eventos (en el método `Game::handleEvents`) delega el manejo de este evento en el objeto `Bow` (llamando a su método `handleEvent`), que actualizará su dirección de movimiento en consecuencia (abajo, arriba o quieto). El arco (en su método `update`) se desplazará siguiendo la dirección establecida. Es importante no realizar la actualización de la posición directamente al pulsar la tecla, sino a través de la dirección, porque si no se desplazará a trompicones.

Generación aleatoria de globos

Para que las partidas no sean todas iguales, la generación de globos debe tener una componente aleatoria. Añadir un globo en la escena supone construir en `Game` un objeto `Balloon` colocado en una determinada posición bajo el borde inferior de la pantalla, con una velocidad y un color determinados. Estos parámetros se pueden escoger aleatoriamente en unos rangos establecidos. Los globos se han de generar uno a uno a una frecuencia constante.

La biblioteca `random` de C++ proporciona distintas clases y métodos para generar números aleatorios (se darán más explicaciones en clase). La clase `Game` debe mantener como atributo un generador de números pseudoaleatorios (por ejemplo, un objeto de la clase `mt19937_64`, utilizando una semilla generada con `random_device()()` o `time(nullptr)`) y utilizar distintas *distribuciones de probabilidad* para generar los números en el rango deseado. Por ejemplo,

- `uniform_int_distribution<int>(a,b)` escoge un número entero con igual probabilidad en el invervalo $[a, b]$, así que podría servir para escoger el color del globo y su posición horizontal.
- `uniform_real_distribution<float>(a,b)` hace lo mismo con un número real, así que podría servir para determinar la velocidad del globo. También se podría utilizar una distribución normal.

Las distribuciones se utilizan como funciones que reciben el generador de números pseudoaleatorios como parámetro. Por ejemplo,

```
int color = uniform_int_distribution<int>(0, 6)(randomGenerator);
```

Manejo de colisiones

La única colisión relevante en este juego es la de las flechas con los globos. La comprobación de colisiones la desencadena la flecha en su movimiento, pasa por el juego y llega hasta cada uno de los globos:

1. el método `update` de `Arrow` construye el rectángulo que delimita su punta () y lo pasa al método `bool checkCollision(const Rectangle&)` de `Game` para comprobar si hay algún obstáculo en su posición actual.
2. el método `checkCollision` llama con el mismo rectángulo al método `bool hit(const Rectangle&)` de cada uno de los globos hasta encontrar uno donde la respuesta sea afirmativa (se puede asumir que no hay dos colisiones por ciclo). Este método debe devolver si se ha detectado una colisión o no, aunque no se haga uso actualmente de esa información.
3. el método `hit` de `Balloon` comprueba si el rectángulo recibido interseca con el globo y en tal caso cambia su estado a explotado (es decir, inicia la animación previa a la destrucción) y devuelve `true`. Para la comprobación de la colisión, se puede utilizar el método `hasIntersection` de la clase `Rectangle`[1].

Eliminación de los objetos del juego

Todos los objetos y recursos del juego se han de eliminar cuando finalice el programa, si bien las flechas y los globos también se crean y destruyen durante la partida. El método `update` de estas clases, además de actualizar su estado, devuelve un valor booleano indicando si el objeto debe seguir activo. Gracias a ello, el método `update` de `Game`, tras llamar al `update` de un objeto, puede saber si ha de destruir el objeto y eliminarlo de la correpondiente vector del juego.

Formato de configuración del juego

El juego depende de diversas constantes que el programador podría fijar en el mismo código, como la frecuencia de aparición de globos, el número inicial de flechas, el rango de velocidades, etc. Sin embargo, cargar esa información desde un archivo es más versátil, ya que permite hacer cambios sin recompilar el programa. En esta práctica vamos a leer un par de opciones de configuración de esta manera.

El formato del archivo de configuración define una propiedad en cada línea, que comienza por una palabra clave a la que sigue una secuencia de parámetros específicos. Las dos opciones reconocidas son:

- `arrows` n indica que n es el número inicial de flechas disponibles.
- `baldelay` t significa que t es el tiempo de espera en milisegundos entre la generación de un globo y su siguiente.

Las líneas que comiencen por una palabra clave no reconocida serán ignoradas sin mostrar ningún aviso,

[1] El método `hasIntersection` es similar a la función `SDL_HasRectIntersectionFloat` de la SDL para `SDL_FRect`s.

utilizando el método `ignore` del flujo de archivo. En particular, la palabra almohadilla (`#`) se usará como inicio de una línea de comentario. El archivo `assets/levels/1.txt` incluido en el material de la práctica obedece a este formato, aunque contiene otras opciones de configuración opcionales que no es necesario procesar. Por ejemplo,

```
# Nivel 1
arrows 7
baldelay 1000
bgcolor 0 170 85
```

El código de la lectura se ha implementar en un método `loadLevel` de `Game`.

Manejo básico de errores

Debes usar excepciones de tipo `string` (con un mensaje informativo del error correspondiente) para los errores básicos que pueda haber. En concreto, es obligatorio contemplar los siguientes tipos de errores: fichero de imagen no encontrado o no válido, fichero de configuración del juego no encontrado, con formato erróneo y error de SDL. Puesto que son todos ellos errores irrecuperables, la excepción correspondiente llegará hasta el `main`, donde deberá capturarse, informando al usuario con el mensaje de la excepción antes de cerrar la aplicación. Recuerda que el tipo del literal `"error"` no es `string` sino `const char*` y has de escribir `string("error")` o `"error"s` en el `throw` para que tenga el tipo adecuado.

Pautas generales obligatorias

A continuación se indican algunas pautas generales que vuestro código debe seguir:

- Asegúrate de que el programa no deje basura revisando la salida de error o la consola de Visual Studio al finalizar el programa. Cuando se compila en modo *Debug*, allí aparecerá un volcado de todos las fugas de memoria detectadas.
- Asegúrate de que el programa hace un correcto uso de la memoria (se inicializan todas las variables y atributos antes de leerlos, no se libera dos veces un mismo bloque de memoria, etc.). El modo de compilación *Sanitize* activa un comprobador de memoria que puede detectar estos errores.
- Todos los atributos deben ser privados excepto quizás algunas constantes del juego definidas como atributos estáticos.
- Define las constantes que sean necesarias, mejor con `constexpr` que con `const`. En general, no deben aparecer literales que pudiesen corresponder con configuraciones del programa en el código.
- No debe haber métodos que superen las 30-40 líneas de código.
- Escribe comentarios claros en el código y explica especialmente los métodos o clases nuevos que hayas añadido. Sé cuidadoso también con los nombres que eliges para variables, parámetros, atributos y métodos. Es importante que denoten realmente lo que son o hacen. Usa preferiblemente nombres en inglés.

Funcionalidades opcionales

1. Implementar una clase `InfoBar` cuyo método `render` se encargue de mostrar en la ventana SDL una barra de información del juego que incluya al menos el número de vidas restantes.
2. Haz que el periodo entre la creación de dos globos consecutivos sea también aleatorio, por ejemplo, con una distribución exponencial (`exponential_distribution`). El atributo `baldelay` será entonces la tasa de disparo de la distribución.
3. Añade soporte para cargar más opciones de configuración desde archivo como las que ya aparecen en los niveles de ejemplo.
4. Implementa el soporte para múltiples niveles: cuando el jugador llegue a una determinada puntuación en el nivel actual n, comprueba si existe existe el archivo $n+1$`.txt` en la carpeta `levels` y carga el nivel correspondiente. El jugador gana la partida si supera todos los niveles disponibles sin interrupción.

5. Haz que la velocidad de la flecha dependa del tiempo que lleva cargada de la forma que se prefiera (por ejemplo, monótona, cíclicamente, etc.). Se puede añadir una visualización de la velocidad para saber cuándo disparar.

Entrega

En la tarea *Entrega de la práctica 1* del campus virtual y dentro de la fecha límite (ver junto al título), cualquiera de los miembros del grupo debe subir el código de la práctica siguiendo las instrucciones que allí aparezcan. La raíz de la solución debe incluir un archivo `info.txt` con los nombres de los componentes del grupo y unas líneas explicando las funcionalidades opcionales incluidas y/o las cosas que no estén funcionando correctamente.

Además, para que la práctica se considere entregada, deberá pasarse una defensa en la que el profesor comprobará, con los dos autores de la práctica, su funcionamiento en ejecución, y si es correcto realizará preguntas (posiblemente individuales) sobre la implementación. Se darán detalles más adelante sobre las fechas, forma y organización de las entrevistas.

Entrega intermedia el 20 de octubre: un 20% de la nota se obtendrá el día 20 de octubre en función de vuestros avances. Para ello debéis mostrar el funcionamiento de vuestra práctica ese mismo día en la sesión de laboratorio. Para obtener la máxima nota en esta revisión se espera que vuestra práctica cargue todos los objetos del juego, aunque no se maneje el movimiento de la bola.